PROJECT 4

Credit Card Default Risk Intelligence & Prediction System

Student Project Guide

Project Type	End-to-End AI / Data Science Project
Project Journey	Foundations → ML → Deep Learning & GenAI → Deployment
Main Problem	Predict customers likely to default on their next credit-card payment
Final Solution	Risk model + ANN + GenAI explanation + API + Streamlit + Docker

1. Project Overview

You are working as a Data Scientist / Risk Analytics Engineer for a financial-services company. The company wants to identify customers who may default on their next credit-card payment so that risk teams can prioritize early intervention.

This project continues through all four skill levels. Students first understand customer and payment data, then build classical ML models, compare them with deep learning, add a responsible GenAI explanation layer, and finally deploy the solution.

2. Business Problem

- Understand customer credit and repayment behavior.
- Identify patterns associated with future default.
- Clean and validate demographic, credit-limit, payment-status, bill and payment-amount data.
- Estimate the probability that a customer will default next month.
- Reduce costly missed high-risk customers while controlling false alarms.
- Explain model results to a risk analyst in understandable language.
- Deploy the final risk-prediction pipeline as a usable application.

3. Project Journey — 4 Skill Levels

LEVEL 1 FOUNDATIONS	LEVEL 2 CORE ML	LEVEL 3 DL + GENAI	LEVEL 4 AI SOLUTION
Clean data EDA + Statistics Preprocessing	Classification Evaluation Tuning + Model Saving	ANN ML vs DL GenAI explanation	FastAPI Streamlit Docker + Cloud

4. Recommended Dataset

Recommended primary source: the UCI Machine Learning Repository dataset 'Default of Credit Card Clients'. UCI lists it as a classification dataset with 30,000 instances and 23 features. It contains credit-limit information, demographic variables, repayment-status history, bill statements and previous payments, with the target indicating default on the next payment. 毅cite孿turn0search5絛

A Kaggle mirror of the same dataset is also available and is listed as CC0/Public Domain. The Kaggle description includes repayment status for six periods, six bill-amount variables and six payment-amount variables. 毅cite孿turn0search15絛

For teaching, prefer the official UCI source as the canonical reference and use Kaggle only as a convenient download/mirror. Record the exact source, version and license in the README.

5. Example Data Dictionary

Column / Group	Meaning	Type	Student Question
LIMIT_BAL	Amount of credit given	Numeric	How does credit limit relate to default?
SEX / EDUCATION / MARRIAGE / AGE	Customer demographic information	Categorical/Numeric	Are there unusual codes or distributions?
PAY_0 … PAY_6	Repayment status over six periods	Ordinal	Does payment delay strongly relate to default?

BILL_AMT1 … BILL_AMT6	Bill statement amounts	Numeric	How does outstanding balance behave?
PAY_AMT1 … PAY_AMT6	Previous payment amounts	Numeric	Are customers paying regularly?
default.payment.next.month	Next-month default indicator	Binary target	What is the class balance?

LEVEL 1 — FOUNDATIONS

Objective

Turn raw customer credit-card data into a clean, trustworthy and analysis-ready dataset. Level 1 ends with EDA, statistical analysis and preprocessing — NOT ML training.

Step 1 — Understand the Business Problem

1. What exactly is being predicted?
2. When is the prediction supposed to be made?
3. What information is available before the next-month outcome?
4. What does default = 1 and default = 0 mean?
5. What business action could follow a high-risk prediction?
6. What could cause data leakage?

Step 2 — Understand the Dataset

- Load the data with Pandas.
- Inspect shape, head, info, dtypes and unique values.
- Check missing values and duplicates.
- Understand the coding of education, marriage, repayment-status variables and target.
- Create a complete data dictionary.
- Separate identifier, demographic, credit, repayment, bill, payment and target columns.

Step 3 — Data Cleaning

- Check missing values and decide how to handle them.
- Investigate duplicate records and identifier issues.
- Validate categorical codes against the dataset documentation.
- Check impossible or suspicious age, credit-limit and amount values.
- Check whether zero values are valid or indicate missingness in context.
- Document every cleaning decision.

Step 4 — Feature Engineering

- Total bill amount across available periods.
- Total payment amount across available periods.
- Average monthly bill amount.
- Average monthly payment amount.
- Payment-to-bill ratio where mathematically and business-wise meaningful.
- Maximum recent payment delay.
- Number of periods with payment delay.
- Credit utilization-style features when the available fields support them.
- Trend features such as whether payments are improving or worsening.

Do not blindly create dozens of features. Every engineered feature must have a clear interpretation and must use only information available at prediction time.

Step 5 — Outlier Analysis

- Use box plots and descriptive statistics.
- Use IQR or Z-score where appropriate.
- Investigate extreme credit limits, bills and payments.

- Check whether large values are legitimate high-value customers or data errors.
- Consider log transformation or robust methods when appropriate.
- Document why values were retained, transformed or removed.

Golden rule: an extreme customer value is not automatically an error.

Step 6 — Exploratory Data Analysis

- Default vs non-default distribution.
- Default rate by repayment status.
- Default rate by credit limit bands.
- Default rate by age groups.
- Default rate by education/marital-status categories.
- Bill amount distributions.
- Payment amount distributions.
- Payment delay distributions.
- Correlation among numerical variables.
- Relationships between payment behavior and default.
- Use groupby, aggregation and pivot-style summaries.

Step 7 — Statistics & Hypothesis Testing

Example question: 'Is the average recent payment amount different between customers who default and customers who do not default?'

- Define H0 and H1.
- Select an appropriate statistical test.
- Run the test with SciPy.
- Interpret the p-value.
- Explain statistical significance versus practical/business significance.

Step 8 — Preprocessing

- Separate X and y.
- Choose categorical/ordinal/numerical treatment carefully.
- Encode variables where required.
- Scale features for algorithms that need scaling.
- Keep the preprocessing reproducible.
- Ensure no future/outcome information enters the feature set.

Level 1 Deliverables

- 01_data_understanding.ipynb
- 02_data_cleaning.ipynb
- 03_eda.ipynb
- 04_statistics.ipynb
- 05_preprocessing.ipynb
- Cleaned dataset
- Data dictionary
- EDA report
- Statistical testing report
- Feature-engineering document
- Preprocessing pipeline

- No ML model

LEVEL 2 — CORE MACHINE LEARNING

Objective

Build a reliable binary-classification risk model, compare algorithms, tune the strongest candidates and save a reproducible prediction pipeline.

Step 1 — Define the Target

Target: next-month default indicator. The project is a binary classification problem.

First inspect class imbalance. A model can achieve high accuracy while still performing poorly on the minority/default class.

Step 2 — Classification Models

- Logistic Regression — interpretable baseline.
- Decision Tree Classifier.
- Random Forest Classifier.
- Gradient Boosting Classifier.
- XGBoost, if included in the syllabus.

Step 3 — Evaluation Metrics

Metric	Meaning	Risk question
Accuracy	Overall correct predictions	Is it hiding class imbalance?
Precision	Correct predictions among predicted defaulters	How many flagged customers are truly high-risk?
Recall	Detected defaulters among all actual defaulters	How many risky customers are we missing?
F1-score	Balance of precision and recall	Can we balance missed risk and false alarms?
ROC-AUC	Separation ability across thresholds	How well does the model rank risk?
PR-AUC	Precision/recall performance across thresholds	Is it useful for an imbalanced target?
Confusion Matrix	TP/TN/FP/FN counts	What type of error is occurring?

Step 4 — Probability & Threshold Thinking

The business may care about risk probability, not only 0/1 labels. Students should understand that changing the classification threshold changes precision and recall.

- Generate predicted probabilities.
- Plot or inspect precision-recall trade-offs.
- Discuss the cost of missing a true defaulter versus unnecessarily flagging a non-defaulter.
- Select a threshold using a justified business objective.

Step 5 — Cross-Validation & Hyperparameter Tuning

- Use stratified cross-validation where appropriate.
- Use GridSearchCV or RandomizedSearchCV.
- Tune important parameters.
- Compare models consistently.
- Check whether improvement is stable rather than a lucky split.

Step 6 — Model Selection & Saving

- Create a model comparison table.
- Select the best model based on business-relevant metrics.
- Save preprocessing + model together when possible.
- Save the model with joblib.
- Record features, threshold, metrics and model version.

Level 2 Deliverables

- 06_machine_learning.ipynb
- Baseline model
- Multiple classification models
- Confusion matrices
- Metrics comparison
- ROC/PR analysis
- Cross-validation results
- Hyperparameter tuning
- Threshold analysis
- Final model selection
- Saved prediction pipeline

LEVEL 3 — DEEP LEARNING & GENAI

Objective

Build a feed-forward ANN, compare it with classical ML and create a responsible GenAI risk-explanation assistant.

Part A — ANN

- Prepare numerical inputs for TensorFlow/Keras.
- Build a feed-forward neural network.
- Choose hidden layers and activation functions.
- Train with validation data.
- Monitor training and validation loss.
- Use early stopping/dropout where appropriate.
- Evaluate the ANN on unseen data.
- Compare ANN against the strongest Level 2 model.

Part B — ML vs DL

- Which model has better recall?
- Which model has better precision?
- Which model ranks risk better?
- Which model is easier to explain?
- Which model is faster to train?
- Does the ANN actually provide enough improvement to justify additional complexity?

Part C — GenAI Risk Explanation Assistant

The trained model remains the source of the numerical risk prediction. GenAI is used to communicate the verified result, not to invent a risk score.

- User enters or selects a customer profile.
- Model returns default probability and class.
- Optional feature-importance/explanation information is passed to the GenAI layer.
- GenAI summarizes the major model factors in simple language.
- GenAI explains limitations and uncertainty.
- GenAI can answer general questions about payment behavior and model interpretation.

Example questions:

- Why was this customer classified as high risk?
- Which recent payment behaviors influenced the prediction?
- What does a high default probability mean?
- Why can this prediction be wrong?
- Which factors should a risk analyst review?

Part D — Responsible AI

- Check model performance across relevant demographic groups where appropriate.
- Discuss potential bias and limitations.
- Do not present correlations as causal relationships.
- Do not automatically treat a demographic attribute as a reason for adverse action.

- Use model explanations as decision support, not unquestionable truth.
- Real financial decisions require organizational policy, human review and applicable regulation.

Level 3 Deliverables

- 07_deep_learning.ipynb
- ANN architecture
- Training/validation curves
- ML vs DL comparison
- GenAI explanation demo
- Responsible-AI discussion
- Saved neural-network model

LEVEL 4 — AI SOLUTION & CLOUD

Objective

Turn the risk model into an application that accepts customer information, returns a verified default-risk prediction/probability and presents an understandable explanation.

Step 1 — FastAPI

- Create a /predict endpoint.
- Validate customer input.
- Load the saved preprocessing pipeline and model.
- Generate default probability.
- Apply the selected classification threshold.
- Return prediction, probability and selected explanation fields.
- Add error handling and logging.

Step 2 — Streamlit

- Create a customer-input form.
- Display predicted default probability.
- Display risk category based on the project threshold.
- Show key model explanation information.
- Show a clear educational-use disclaimer.

Step 3 — Docker

- Create Dockerfile.
- Install dependencies.
- Package the API/application.
- Test locally in a container.

Step 4 — Git/GitHub

- Maintain clean commits.
- Keep credentials/API keys out of the repository.
- Document setup and usage.
- Add model/version information to README.

Step 5 — Cloud

- Deploy the API/application to a suitable platform.
- Configure environment variables.
- Test the deployed endpoint.
- Document deployment steps.

Final Architecture

Risk Analyst / User → Streamlit UI → FastAPI /predict → Preprocessing Pipeline → Saved Risk Model → Default Probability → Threshold → Risk Result → Explanation Layer → User

Suggested Repository Structure

```
credit-card-default-ai/
│
├── data/
```

```
│   ├── raw/
│   └── processed/
│
├── notebooks/
│   ├── 01_data_understanding.ipynb
│   ├── 02_data_cleaning.ipynb
│   ├── 03_eda.ipynb
│   ├── 04_statistics.ipynb
│   ├── 05_preprocessing.ipynb
│   ├── 06_machine_learning.ipynb
│   └── 07_deep_learning.ipynb
│
├── src/
│   ├── preprocessing.py
│   ├── features.py
│   └── prediction.py
│
├── models/
│   ├── best_model.joblib
│   └── neural_network.keras
│
├── api/
│   └── main.py
│
├── app/
│   └── app.py
│
├── reports/
├── Dockerfile
├── requirements.txt
└── README.md
```

Technology Stack

- Python
- NumPy
- Pandas
- Matplotlib
- Seaborn
- SciPy
- Scikit-learn
- TensorFlow/Keras
- FastAPI
- Streamlit
- Git/GitHub
- Docker
- Cloud Platform
- Optional: SHAP

Final Project Outcome

By the end of Project 4, students should demonstrate the complete journey from raw credit-card data to a deployable AI risk-intelligence solution.

7. Understand the risk/business problem.
8. Inspect and clean customer credit data.
9. Perform EDA and statistical testing.
10. Engineer meaningful repayment-behavior features.
11. Build reproducible preprocessing.
12. Train multiple classification models.
13. Evaluate with precision, recall, F1, ROC-AUC and PR-AUC.
14. Perform cross-validation and hyperparameter tuning.
15. Choose and save a final model and threshold.
16. Build and compare an ANN.
17. Create a responsible GenAI explanation layer.
18. Expose the model through FastAPI.
19. Build a Streamlit interface.
20. Containerize using Docker.
21. Publish and document the project in GitHub.
22. Deploy the application where possible.

How Students Should Think

Do not start by asking: 'Which algorithm should I use?'

23. Business problem → What risk are we trying to identify?
24. Prediction time → What information is available at that moment?
25. Data → What does every variable mean?
26. Quality → Can we trust the values?
27. EDA → What repayment patterns do we see?
28. Statistics → Which observations have evidence?
29. Features → What behavior can safely be used?
30. Model → Which classifier fits the objective?
31. Evaluation → Which mistakes matter most?
32. Probability → How should risk scores be interpreted?
33. Explanation → Can an analyst understand the result?
34. Deployment → How will the model be consumed and monitored?

Submission Checklist

- Problem statement
- Dataset source + license/attribution
- Data dictionary
- Cleaning report
- EDA report
- Hypothesis test(s)
- Feature engineering report

- Preprocessing pipeline
- Model comparison
- Confusion matrices
- Precision/Recall/F1
- ROC-AUC / PR-AUC
- Cross-validation
- Hyperparameter tuning
- Threshold analysis
- Saved model
- ANN comparison
- GenAI demo
- Responsible AI discussion
- FastAPI endpoint
- Streamlit application
- Dockerfile
- GitHub repository
- README
- Deployment demonstration

Viva Questions

- Why is default prediction a classification problem?
- What is the target variable?
- Why can accuracy be misleading for default prediction?
- What is class imbalance?
- What is a false negative in this project?
- Why might recall be important?
- What is precision?
- What is ROC-AUC?
- When is PR-AUC useful?
- Why do we use predicted probabilities?
- What happens when we change the classification threshold?
- What is data leakage in a credit-risk project?
- Why should future payment information not be used?
- Why use cross-validation?
- Why might Random Forest outperform Logistic Regression?
- Why might an ANN not improve a tabular ML problem?
- What is the role of GenAI?
- How would you explain a prediction to a risk analyst?
- How would you monitor model drift?
- How would you check for unfair model behavior?

Golden Rules

- Business problem first, algorithm second.
- Level 1 ends with clean data + EDA + statistics + preprocessing — not ML.
- Outlier does not automatically mean error.

- Never use future/outcome information as an input feature.
- Never fit preprocessing on the test set.
- Do not judge a risk model using accuracy alone.
- Probability and threshold are important parts of classification.
- Correlation does not prove causation.
- GenAI explains verified model outputs; it does not invent risk scores.
- Financial AI requires responsible use, monitoring and human oversight.

One-Line Project Definition

Build an end-to-end credit-card default risk intelligence system that analyzes repayment behavior, predicts next-month default probability with ML/DL, explains the result responsibly with GenAI, and serves the model through a deployable application.